

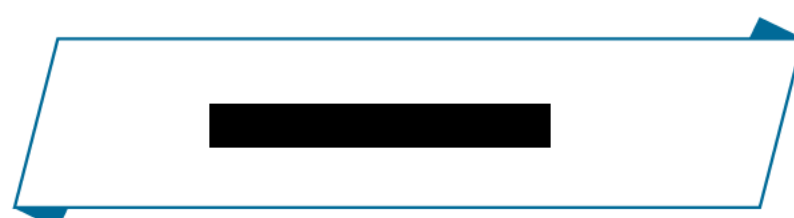
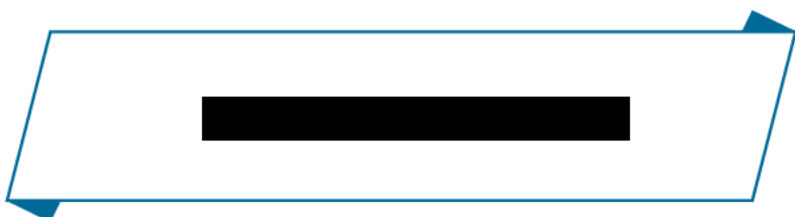
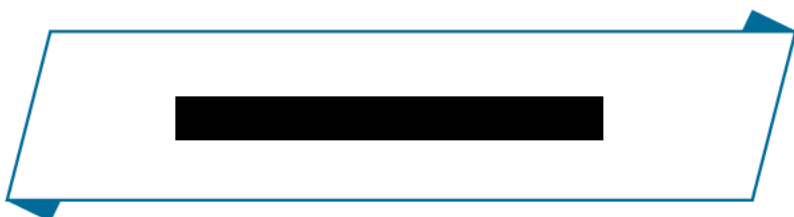
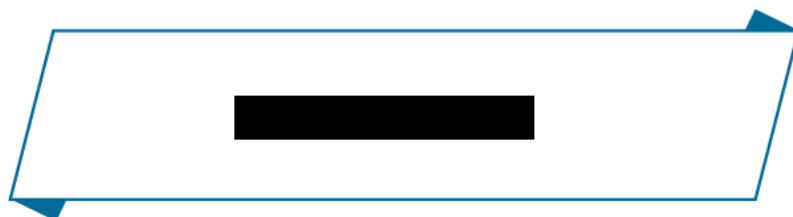
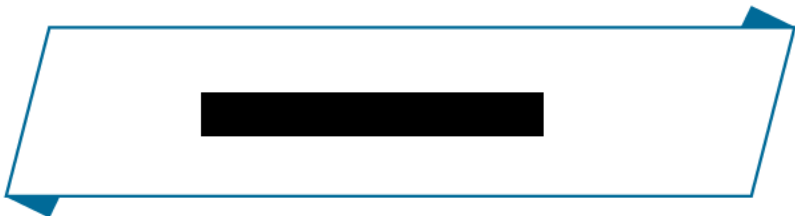
基于卷积神经网络的 卷积码编码识别

汇报小组:

时间:

2025

分工



目录

CONTENTS

01 一、卷积码生成多项式

02 二、卷积神经网络原理

03 三、系统程序流程与运行结果

04 四、总结

05 五、参考文献

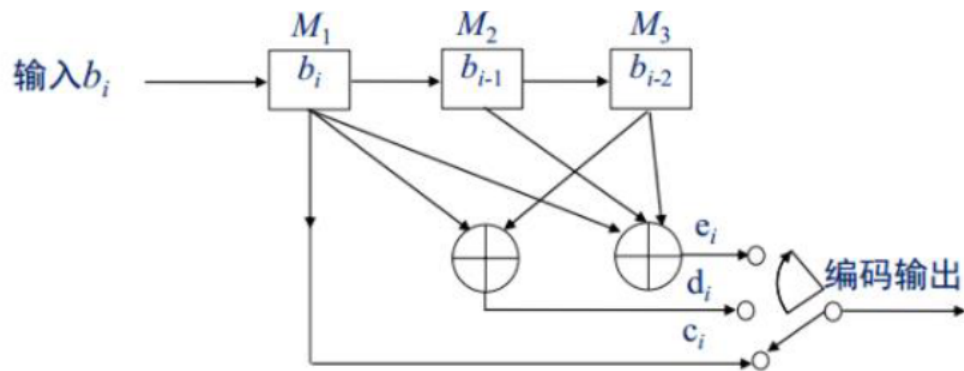
Part 01

一、卷积码生成多项式

卷积码生成多项式

卷积码是一种差错控制编码，由P.Elias于1955年发明。因为数据与二进制多项式滑动相关故称卷积码。卷积码在通信系统中应用广泛，如IS-95，TD-SCDMA，WCDMA，IEEE 802.11及卫星等系统中均使用了卷积码。

由于我们课上接触过卷积码的基本原理，这里不做过多赘述，直接介绍卷积码生成多项式的一个示例：



生成多项式：

$$\begin{cases} g_1(x) = 1 \\ g_2(x) = 1 + x^2 \\ g_3(x) = 1 + x + x^2 \end{cases}$$

假设输入序列[1101]，其多项式： $m(x) = 1 + x + x^3$ (注意最左侧的比特对应多项式的最低次)。则：

$$\begin{cases} y_1(x) = m(x)g_1(x) = 1 + x + x^3 \\ y_2(x) = m(x)g_2(x) = 1 + x + x^2 + x^5 \\ y_3(x) = m(x)g_3(x) = 1 + x^4 + x^5 \end{cases}$$

根据卷积码的定义，输出应为 $4 \times 3 = 12$ 位，故选取前12位输出：(111,110,010,100)

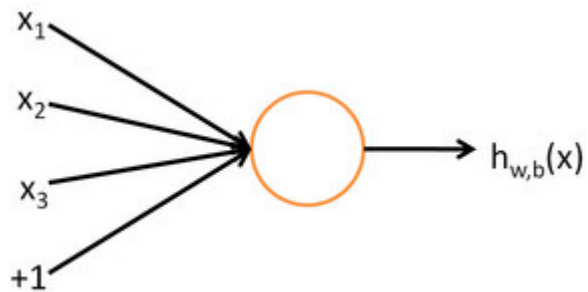
Part 02

二、卷积神经网络原理

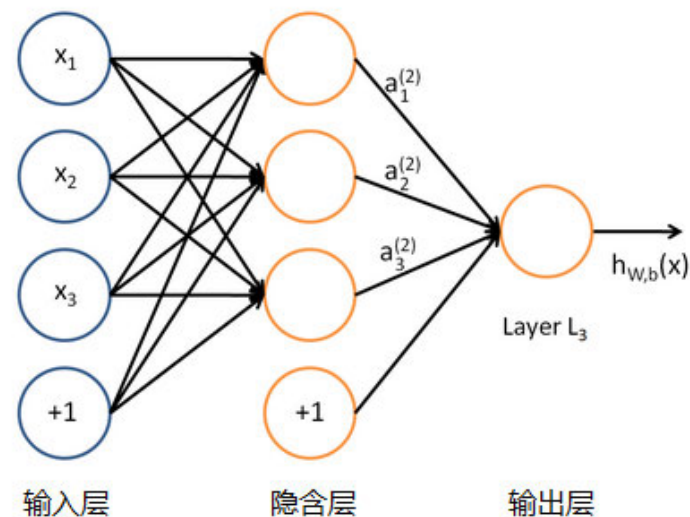
神经网络

首先什么是神经网络呢？神经网络也指的是**人工神经网络**（Artificial Neural Networks，简称ANNs），是一种模仿生物神经网络行为特征的算法数学模型，由**神经元、节点与节点之间的连接（突触）**所构成。

每个神经网络单元抽象出来的数学模型如下，也叫**感知器**，它接收多个输入（ x_1 ， x_2 ， $x_3 \dots$ ），产生一个输出，这就好比是神经末梢感受各种外部环境的变化（外部刺激），然后产生电信号，以便于转导到神经细胞。如下图所示：



单个的感知器就构成了一个简单的模型，但在现实世界中，实际的决策模型则要复杂得多，往往是由多个感知器组成的多层网络，如下图所示，这也是经典的神经网络模型，由**输入层、隐含层、输出层**构成。如下图所示：



人工神经网络可以映射任意复杂的非线性关系，具有很强的鲁棒性、记忆能力、自学习等能力，在分类、预测、模式识别等方面有着广泛的应用。

卷积

卷积是一种有效提取图片特征的方法。一般用一个正方形卷积核 W :

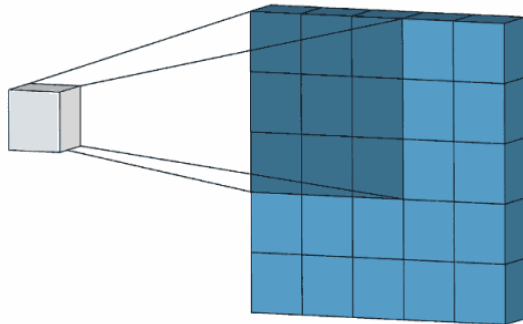
$$W = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \dots & \dots & \dots & \dots \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{bmatrix}_{(m \times n)}$$

遍历图片上每一个像素点。图片与卷积核重合区域内相对应的每一个像素值，乘卷积核内相对应点的权重，然后求和，再加上偏置后，公式为：

$$z = w_1x_1 + w_2x_2 + \dots + w_{mn}x_{mn} = \sum_{k=1}^{mn} w_kx_k = W^T X$$

最后得到输出图片中的一个像素值。

一幅图像的一个完整的卷积运算过程示意图：



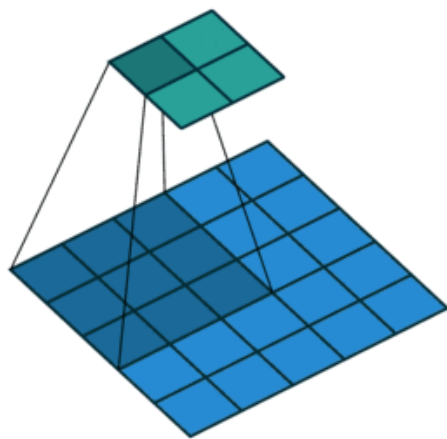
步长

步长是滑动一定的间距，但这个间距该如何定义呢？这个概念就是卷积的“步长”（stride）。经过步长 s 的操作后，其输出图像的尺寸为：

$$\left\lfloor \frac{n - f}{s} \right\rfloor + 1$$

其中 n 是图像大小， f 是卷积核大小， s 是步长。

stride = 1 表示卷积核滑过每一个相距是 1 的像素，是最基本的单步滑动，作为标准卷积模式。Stride 是 2 表示卷积核的移动步长是 2，跳过相邻像素，输出图像缩小为原来的 1/2。Stride 是 3 表示卷积核的移动步长是 3，跳过 2 个相邻像素，图像缩小为原来的 1/3，以此类推。



填充

在标准的卷积过程中，存在两个问题：

- 每次卷积运算后，图像就会缩小尺寸。在经历多次运算后，图像最终会失去其本来的形状，变为 1×1 的“柱状”。
- 对于图像边缘的像素，只被一个输出使用，但图像中间的像素，则被多个输出使用。这意味着卷积过程丢掉了图像边缘位置的许多信息。

对于这个问题，可以采用额外的“假”像素（通常值为 0，因此经常使用的术语“零填充”）填充边缘。这样，在滑动时的卷积核可以允许原始边缘像素位于其中心，同时延伸到边缘之外的假像素。假设填充的像素大小为 p ，则 n 就变成了 $n + 2p$ ，故其输出图像的尺寸为 $\left\lceil \frac{n+2p-f}{s} \right\rceil + 1$ 。

至于是否选择填充像素，通常有两个选择，分别叫做 Valid 卷积和 Same 卷积。

- Valid 卷积意味着不填充，即图像会通过卷积并逐渐缩小，输出的图像尺寸即为上述公式： $\left\lceil \frac{n+2p-f}{s} \right\rceil + 1$ 。
- Same 卷积意味着填充，输出图像的尺寸与输入图像的尺寸相同。

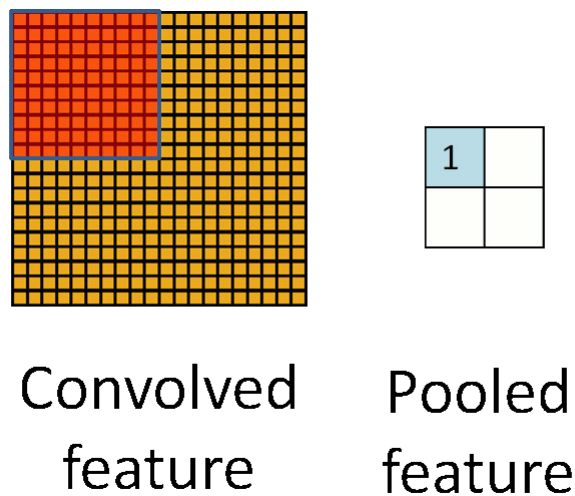
根据上述尺寸的计算公式，令 $\left\lceil \frac{n+2p-f}{s} \right\rceil + 1 = n$ ，可得到 $p = \frac{(n-1)s-n+f}{2}$ 。当 $s=1$ 时， $p = \frac{f-1}{2}$ 。

池化

随着模型网络不断加深，卷积核越来越多，要训练的参数还是很多，而且直接拿卷积核提取的特征直接训练也容易出现过拟合的现象。CNN使用的另一个有效的工具被称为“池化(Pooling)”出现并解决了上面这些问题，为了有效地减少计算量，池化就是将输入图像进行缩小，减少像素信息，只保留重要信息；为了有效地解决过拟合问题，池化可以减少数据，但特征的统计属性仍能够描述图像，而由于降低了数据维度，可以有效地避免过拟合。

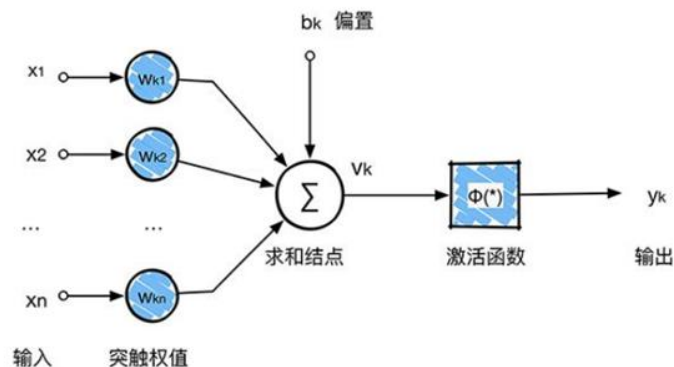
给出池化的定义，对不同位置区域提取出有代表性的特征（进行聚合统计，例如最大值、平均值等），这种聚合的操作就叫做池化，池化的过程通常也被称为特征映射的过程（特征降维）。

池化的过程如下图所示：



激活函数

为了使神经网络能够拟合出各种复杂的函数，必须使用非线性激活函数，用来加入非线性因素，把卷积层输出结果做非线性映射。



在神经网络的正向传播过程中 ($Z[l]=W[l]A[l-1]+b[l]$, $A[l]=g(Z[l])$)，如果我们将激活函数 g 去掉，则 $A[l]=Z[l]$ ，这个有时被叫做线性激活函数（恒等激励函数），具体公式如下：

第一层： $A[1]=Z[1]=W[1]X+b[1]$

第二层： $A[2]=Z[2]=W[2]A[1]+b[2]$

将 $A[1]$ 带入可得第二层： $A[2]=Z[2]=W[2](W[1]X+b[1])+b[2]=W[2]W[1]X+W[2]b[1]+b[2]$ ，令 $W[2]W[1]=W'$ ， $W[2]b[1]+b[2]=b'$ ，则第二层变为： $A[2]=Z[2]=W'X+b'$ 。

依此类推，网络的输出仅仅只是输入特征的线性组合。实际上，无论网络有多少层，整体完全可以仅使用1层表示。同理，引入其他线性函数（如 $g'=2x$ ）仍然起不到任何作用，因为线性函数的组合本身仍是线性函数。

常用的激活函数有sigmoid、tanh、relu等等，前两者sigmoid / tanh比较常见于全连接层，后者ReLU常见于卷积层。

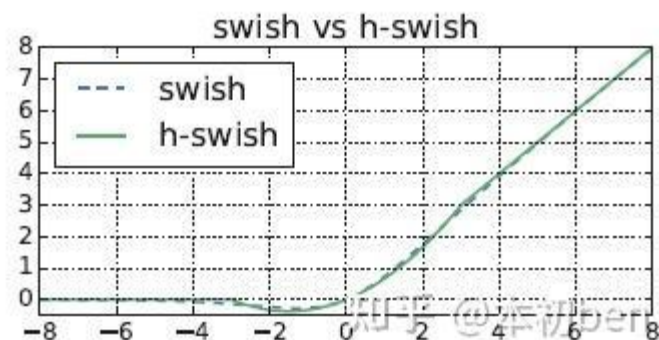
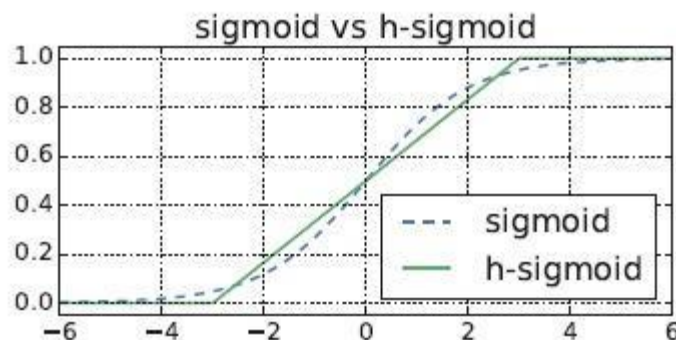
sigmoid函数

sigmoid函数也叫Logistic函数，其公式如下：

$$S(x) = \frac{1}{1 + e^{-x}}$$

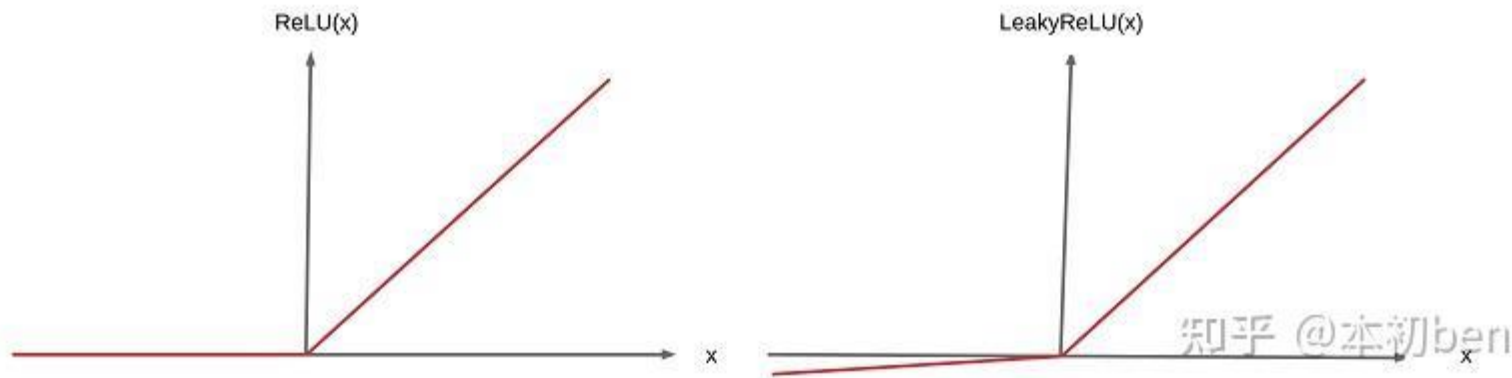
sigmoid函数的优点是平滑、易于求导，缺点是激活函数计算量大，反向传播求误差梯度时，求导涉及除法，反向传播时，很容易就会出现梯度消失的情况，从而无法完成深层网络的训练。

为了解决sigmoid计算量大对低性能硬件不友好的问题，有研究人员针对此专门设计了近似的硬件友好的函数 [h-sigmoid](#)和[h-swish](#)函数，这里的h指的就是hardware的意思。



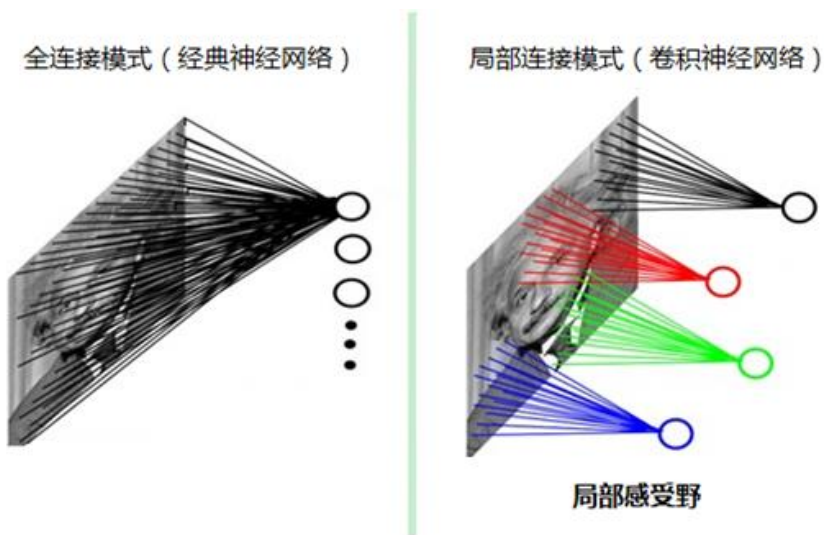
ReLU函数

这里主要指常用的ReLU，ReLU6和leaky ReLU。ReLU比较好部署，小于0的部分为0，大于0的部分为原始值，只需要判断一下符号位就行；ReLU6与ReLU相比也只是在正向部分多了个阈值，大于6的值等于6，在实现时多了个比较也不算麻烦；而leaky ReLU和ReLU正向部分一样，都是大于0等于原始值，但负向部分却是等于原始值的1/10，浮点运算的话乘个0.1就好了，如果因为量化要实现整数运算，这块可以做个近似，如0.1用13>>7来代替，具体实现方法多种多样，还算简单。在卷积神经网络中应用广泛。



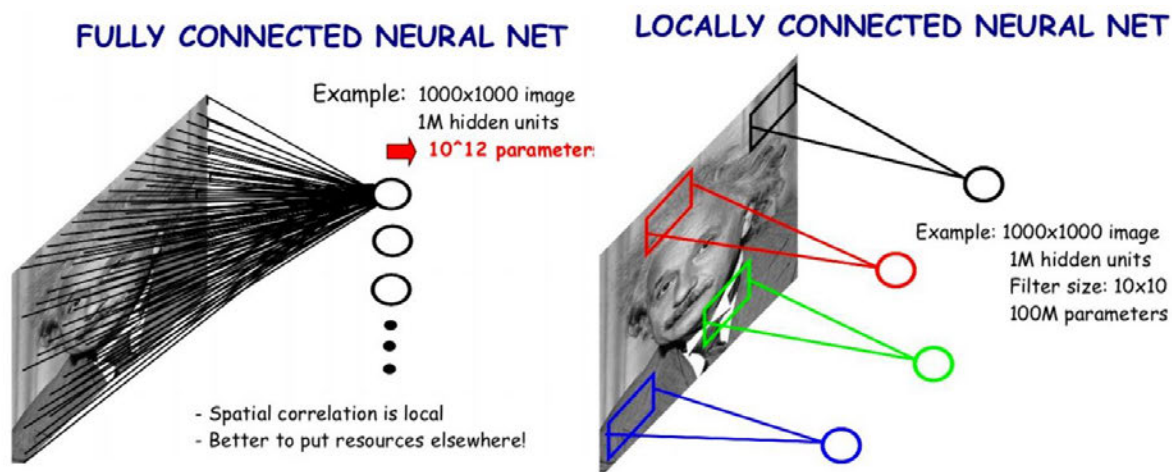
局部感知

如果采用经典的神经网络模型，如下图所示：



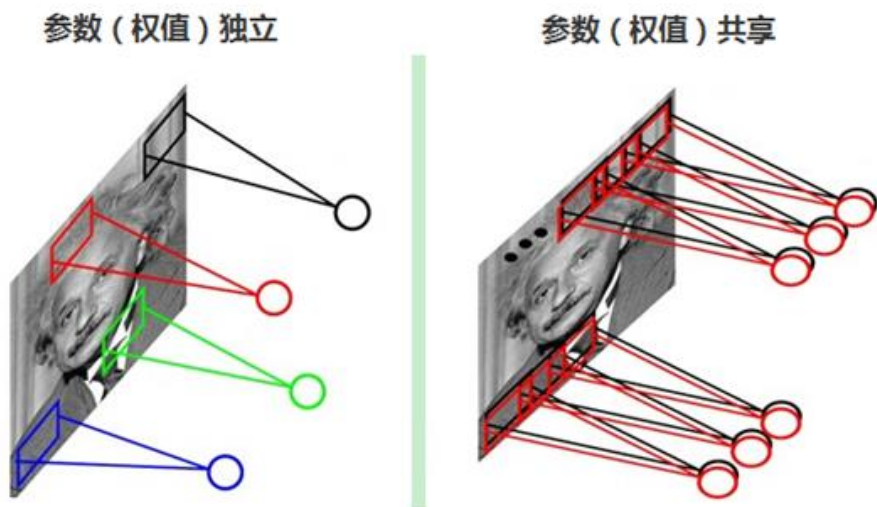
则需要读取整幅图像作为神经网络模型的输入（即全连接的方式），当图像的尺寸越大时，其连接的参数将变得很多，从而导致计算量非常大。比如对于一张1000x1000像素的图片，如果我们有1M隐藏层单元，那么一共有 10^{12} 个参数，这显然是不能接受的。

而人类对外界的认知一般是从局部到全局、从片面到全面，先对局部有感知的认识，再逐步对全体有认知，**这是人类的认识模式**。类似的，在机器识别图像时也没有必要把整张图像按像素全部都连接到神经网络中，局部范围内的像素之间联系较为紧密，而距离较远的像素则**相关性较弱**。因而，每个神经元其实没有必要对全局图像进行感知，只需要对局部进行感知，然后在更高层将局部的信息综合起来就得到了全局的信息。这种模式就是卷积神经网络中降低参数数目的重要神器：**局部感受野**，节省了内存。

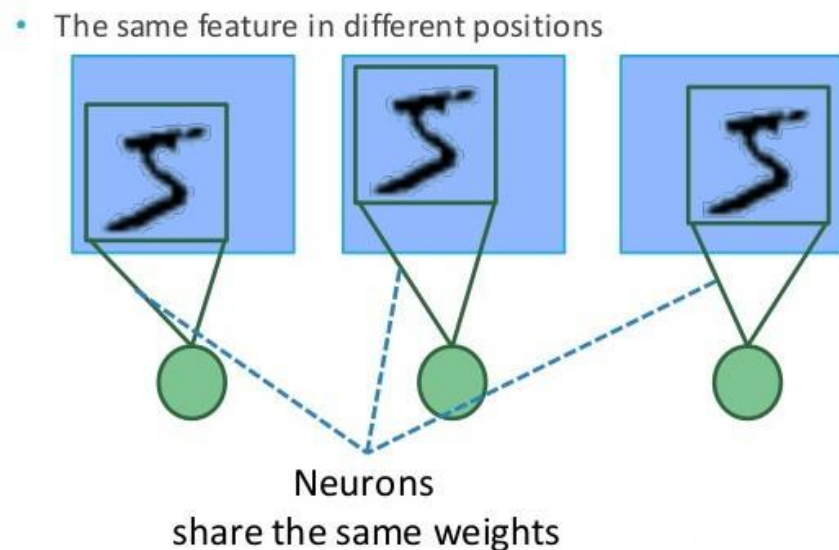


参数（权值）共享

每张自然图像（人物、山水、建筑等）都有其固有特性，也就是说，图像其中一部分的统计特性与其它部分是接近的。这也意味着**这一部分学习的特征也能用在另一部分上**。因此，在局部连接中隐藏层的每一个神经元连接的局部图像的权值参数（例如 5×5 ），将这些**权值参数共享**给其它剩下的神经元使用，那么此时不管隐藏层有多少个神经元，需要训练的参数就是这个局部图像的权重参数（例如 5×5 ），也就是卷积核的大小，这样大大减少了训练参数。如下图所示：



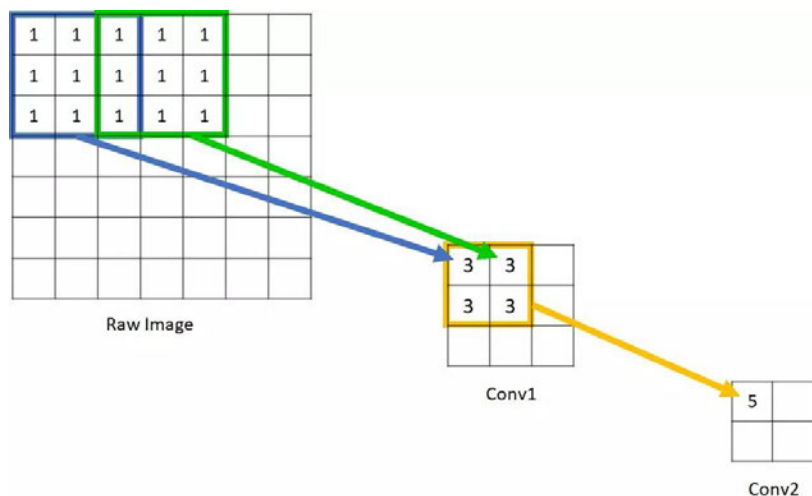
举个例子，假设使用某一特征过滤器（检测器）提取“手写5”的特征，那么图片的右下角区域，以及旁边的各个区域都可以使用这个过滤器。每个特征检测器以及输出都可以在输入图片的不同区域中使用同样的参数，以便提取特征“5”。而对于全连接网络来说，如果模式出现在新的位置，它只能重新学习这个模式。这使得卷积神经网络在处理图像时可以高效利用数据（因为视觉世界从根本上具有**平移不变性**），只需要更少的训练样本就可以学到具有泛化能力的数据表示。



感受野

感受野用来表示网络内部的不同神经元对原图像的感受范围的大小，换句话说，即为每一层输出的特征图上的像素点在原始图像上映射的区域大小。

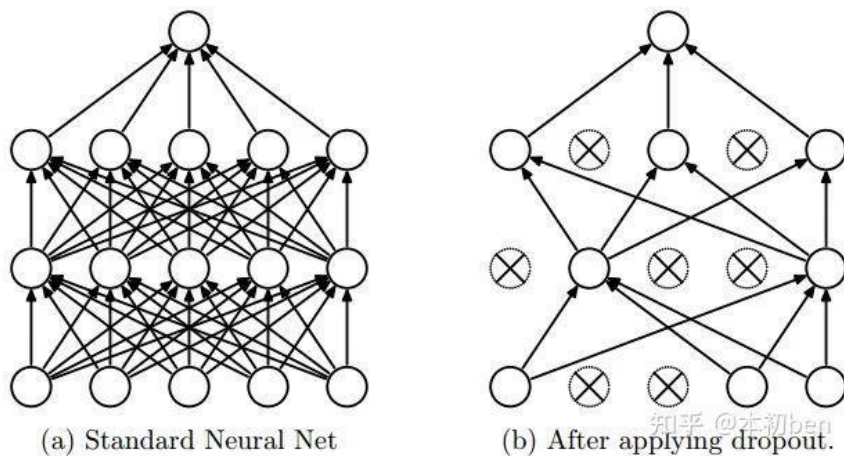
神经元之所以无法对原始图像的所有信息进行感知，是因为在这些网络结构中普遍使用卷积层和pooling层，在层与层之间均为局部连接。神经元感受野的值越大表示其能接触到的原始图像范围就越大，也意味着它可能蕴含更为全局，语义层次更高的特征；相反，值越小则表示其所包含的特征越趋向局部和细节。因此感受野的值可以用来大致判断每一层的抽象层次。



可以看到在Conv1中的每一个单元所能看到的原始图像范围是3x3，而由于Conv2的每个单元都是由 2x2范围的Conv1构成，因此回溯到原始图像，其实是能够看到5x5的原始图像范围的。因此我们说Conv1的感受野是3，Conv2的感受野是5。输入图像的每个单元的感受野被定义为1，这应该很好理解，因为每个像素只能看到自己。

Dropout

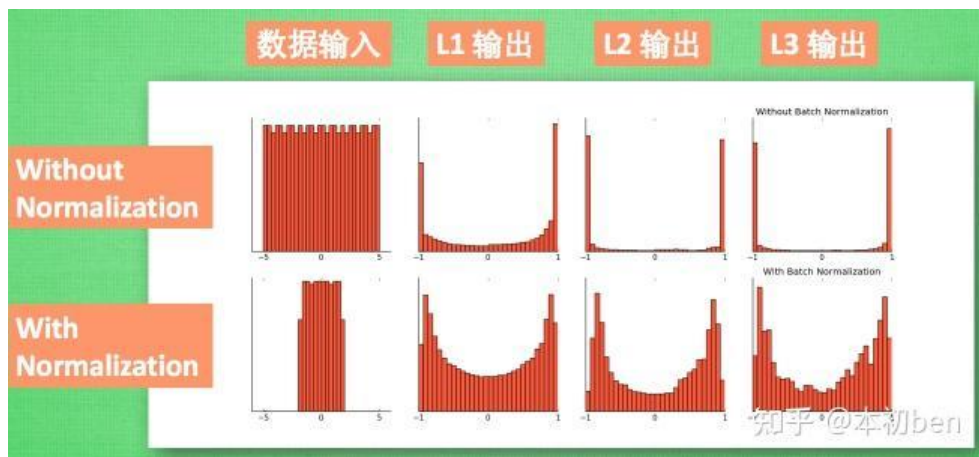
Dropout是指在深度学习网络的训练过程中，对于神经网络单元，按照一定的概率将其暂时从网络中丢弃。注意是暂时，对于随机梯度下降来说，由于是随机丢弃，故而每一个mini-batch都在训练不同的网络。当进行测试和推理时，Dropout将不起作用。



Dropout是CNN中防止过拟合提高效果的一个重要方法。Dropout为什么有效？Dropout背后理念和集成模型很相似。在Dropout层，不同的神经元组合被关闭，这代表了一种不同的结构，所有这些不同的结构使用一个的子数据集并行地带权重训练，而权重总和为1。如果Dropout层有 n 个神经元，那么会形成 2^n 个不同的子结构。在预测时，相当于集成这些模型并取均值。这种结构化的模型正则化技术有利于避免过拟合。Dropout有效的另外一个观点是：由于神经元是随机选择的，所以可以减少神经元之间的相互依赖，从而确保提取出相互独立的重要特征。

Batch Normalization

BN层通常添加在每个神经网络层和激活层之间，对神经网络层输出的数据分布进行统一和调整，变成均值为0方差为1的标准正态分布，使输出位于激活层的非饱和区，达到加快收敛的效果。BN层的效果演示如下图：



BN算法的公式如下图：

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;
Parameters to be learned: γ, β
Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$
$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$
$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$
$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

标准化工序

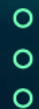
反标准化工序

知乎 @本初ben

前三个公式表述的标准化工序并不难理解，但最后还有一个反标准化工序，即对标准化后的数据再扩展和平移，其中涉及到扩展参数 γ 和平移参数 β ，这两个参数是可学习的，能够让模型在训练过程中自己去修改，这样神经网络就能够自己琢磨出前面的标准化工序有没有起到优化的作用，如果没有，就可以通过调整 γ 和 β 参数来在一定程度上抵消标准化操作。

Part 03

三、系统程序流程与运行结果



卷积编码与数据集生成

我们编写了卷积码编码函数（左）与生成一定误码率的数据集函数（右）

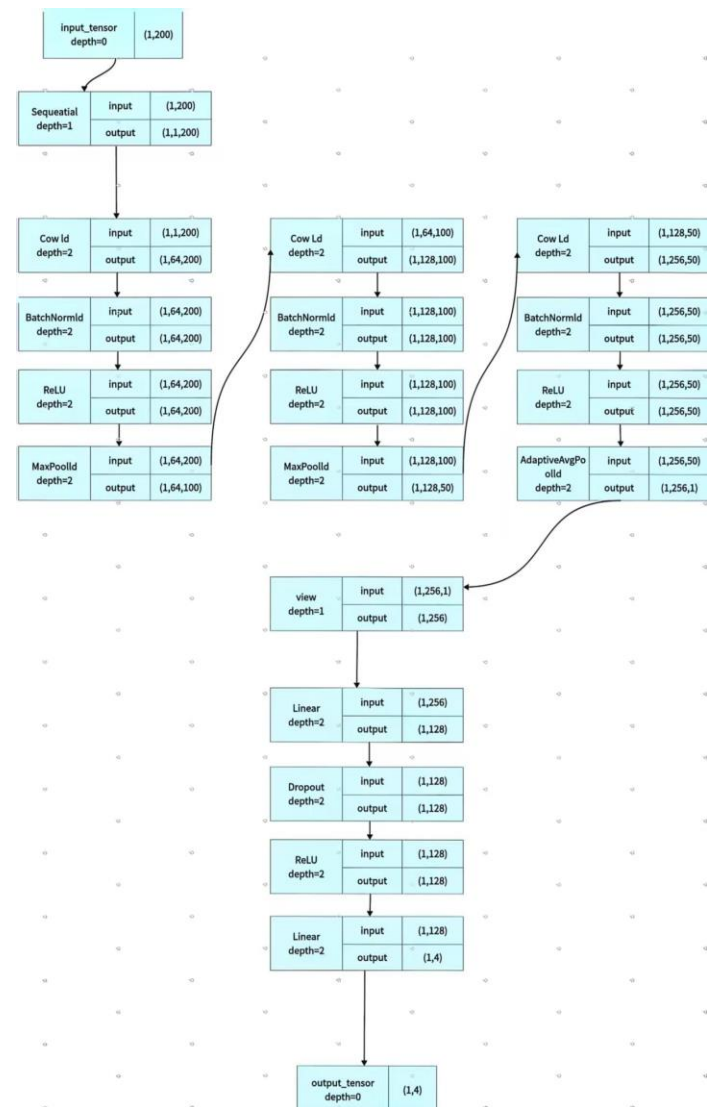
```
def encode(bits, generators, constraint_length=7): 1 个用法
    """
    卷积码编码函数
    :param bits: 输入比特序列（一维数组，0/1值）
    :param generators: 生成器多项式列表（十进制表示）
    :param constraint_length: 约束长度（决定移位寄存器长度，默认7）
    :return: 编码后的比特流（numpy数组，长度为len(bits)*len(generators)）
    """
    encoded = [] # 存储编码后的比特
    shift_register = [0] * constraint_length # 初始化移位寄存器（全0）
    for bit in bits:
        # 更新移位寄存器：新比特插入首位，其余右移（模拟移位操作）
        shift_register = [bit] + shift_register[:-1]
        outputs = [] # 存储当前比特的两个输出
        for g in generators: # 遍历每个生成器多项式
            xor_sum = 0
            for i in range(constraint_length): # 逐位检查生成器掩码
                # 将生成器多项式转换为二进制掩码，确定哪些位参与异或
                mask = (g >> (6 - i)) & 1 # 例如，g=0b1101011对应i从0到6的掩码位
                xor_sum0 = shift_register[i] & mask # 当前寄存器位与掩码位按位与
                xor_sum = xor_sum ^ xor_sum0 # 异或累加
            outputs.append(xor_sum % 2) # 取模2得到输出比特
        encoded.extend(outputs) # 将两个输出比特添加到编码流
    return np.array(encoded) # 转换为numpy数组
```

```
def generate_dataset(ber = -1, num_samples=1000, seq_length=100): 2 用法
    """
    生成训练/测试数据集，包含带噪声的编码序列和对应标签
    :param ber:
    :param num_samples: 样本数量
    :param seq_length: 输入比特序列长度（编码后长度为seq_length*2）
    :return: 特征张量X和标签张量y
    """
    X = []
    y = []
    for _ in range(num_samples):
        # 随机选择标签（0-3）
        label = np.random.randint(low=0, high=4)
        generators = generator_polynomials[label] # 获取对应生成器
        # 生成随机输入比特序列（0/1）
        bits = np.random.randint(0, 2, seq_length)
        encoded = encode(bits, generators) # 编码得到200长度序列（2倍）
        # 模拟信道误码：按ber概率翻转比特
        if ber == -1:
            ber = np.random.uniform(low=0, high=0.1)
        # 生成误码掩码（1表示该位出错）
        mask = np.random.choice(
            a=[0, 1],
            size=encoded.shape,
            p=[1 - ber, ber] # 误码概率为ber
        )
        # 添加误码：异或操作（0+0=0, 0+1=1, 1+1=0）
        corrupted = (encoded + mask) % 2
        X.append(corrupted.astype(np.float32)) # 转换为float32类型
        y.append(label)
    # 转换为PyTorch张量
    return torch.tensor(np.array(X)), torch.tensor(np.array(y))
```

模型结构

神经网络结构(ConvCodeClassifier)分为两部分:

- 特征提取器 (features): 通过卷积层逐级提取局部和全局特征。
- 分类器 (classifier): 将特征映射到特定类别。



模型结构

特征提取器 (features 模块)

1. 第一卷积块:

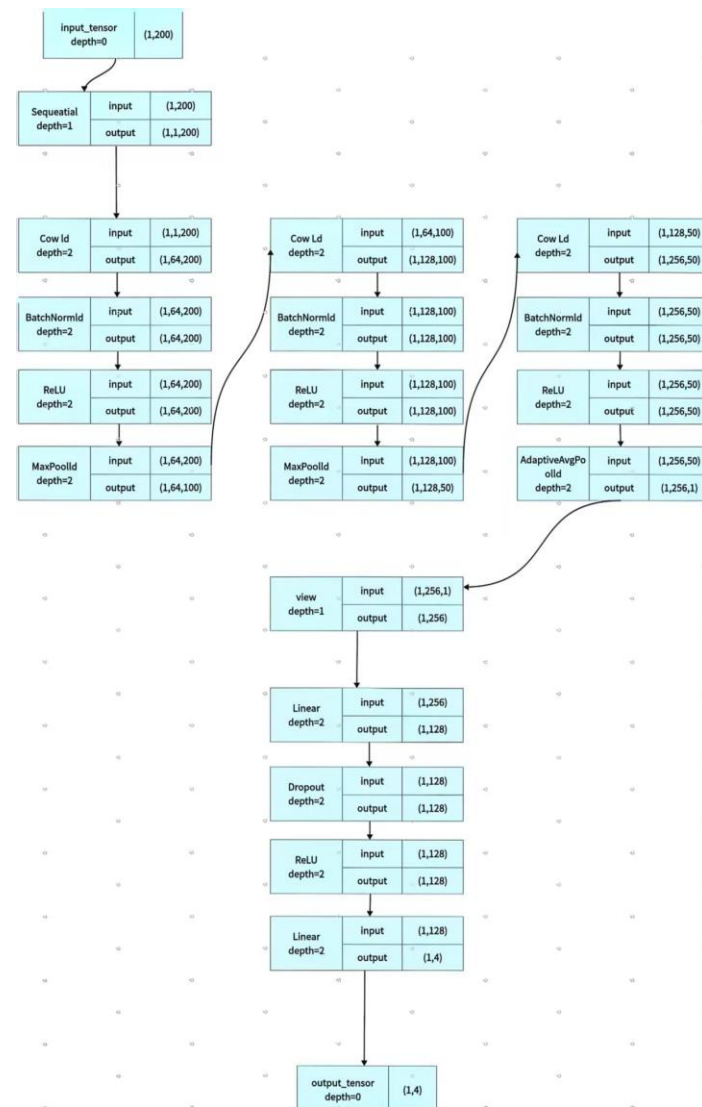
- 大卷积核 (15): 捕捉长距离局部模式 (适合卷积码的约束长度 7)。
- 最大池化: 降维至 100, 保留显著特征, 减少计算量。

2. 第二卷积块:

- 中等卷积核 (9): 提取中等尺度特征。
- 池化: 进一步压缩序列长度至 50, 增强平移不变性。

3. 第三卷积块:

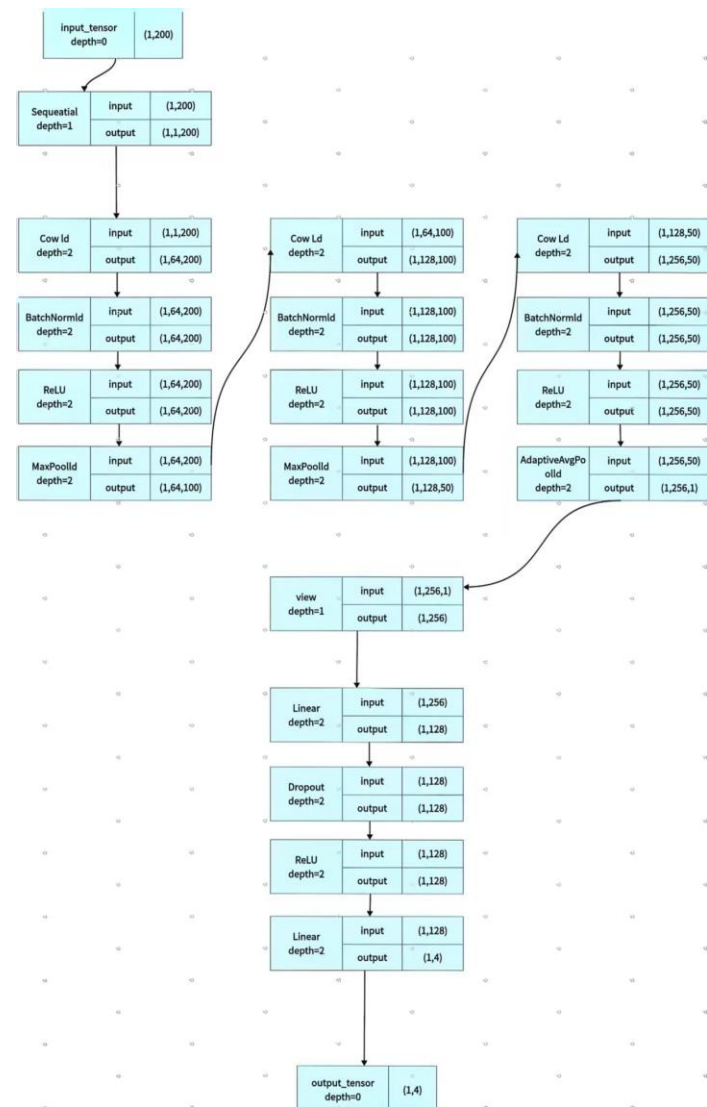
- 小卷积核 (5): 捕捉细粒度局部特征。
- 自适应平均池化: 将每个通道的 50 维特征压缩为 1 个全局均值, 生成 256 维特征向量。



模型结构

分类器 (classifier 模块)

1. 第一层为全连接层，将 256 维特征压缩到 128 维，筛选关键信息。
2. Dropout: 在训练时随机关闭部分神经元，增强泛化能力。
3. 输出层: 映射到 4 维空间，也为全连接层。



模型结构

下图为神经网络的结构代码（左）与训练后的模型的文字描述（右）

```
class ConvCodeClassifier(nn.Module): 5 用法
    """ 卷积神经网络分类器，用于识别不同卷积码生成的序列 """
    def __init__(self, input_length=200):
        super().__init__()
        self.features = nn.Sequential(
            # 第一层卷积：输入通道1，输出64，卷积核15，填充7保持长度
            nn.Conv1d(in_channels=1, out_channels=64, kernel_size=15, padding=7),
            nn.BatchNorm1d(64), # 批归一化
            nn.ReLU(), # 激活函数
            nn.MaxPool1d(2), # 最大池化，长度减半

            # 第二层卷积：输入64，输出128，核9，填充4
            nn.Conv1d(in_channels=64, out_channels=128, kernel_size=9, padding=4),
            nn.BatchNorm1d(128),
            nn.ReLU(),
            nn.MaxPool1d(2), # 长度再减半 → 200/2/2=50

            # 第三层卷积：输入128，输出256，核5，填充2
            nn.Conv1d(in_channels=128, out_channels=256, kernel_size=5, padding=2),
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.AdaptiveAvgPool1d(1) # 自适应平均池化到长度1 → 256维特征向量
        )
        self.classifier = nn.Sequential(
            nn.Linear(in_features=256, out_features=128), # 全连接层
            nn.Dropout(0.5), # Dropout防止过拟合
            nn.ReLU(),
            nn.Linear(in_features=128, out_features=4) # 输出4类（对应4种生成器）
        )

    def forward(self, x):
        # 输入形状：(batch_size, input_length) → 添加通道维度 → (batch_size, 1, input_length)
        x = x.unsqueeze(1)
        x = self.features(x) # 经过卷积层 → (batch_size, 256, 1)
        x = x.view(x.size(0), -1) # 展平 → (batch_size, 256)
        return self.classifier(x) # 分类 → (batch_size, 4)
```

```
===== 模型结构 =====
ConvCodeClassifier(
  (features): Sequential(
    (0): Conv1d(1, 64, kernel size=(15,), stride=(1,), padding=(7,))
    (1): BatchNorm1d(64, eps=1e-05, momentum=0.1, affine=True, track running stats=True)
    (2): ReLU()
    (3): MaxPool1d(kernel size=2, stride=2, padding=0, dilation=1, ceil mode=False)
    (4): Conv1d(64, 128, kernel size=(9,), stride=(1,), padding=(4,))
    (5): BatchNorm1d(128, eps=1e-05, momentum=0.1, affine=True, track running stats=True)
    (6): ReLU()
    (7): MaxPool1d(kernel size=2, stride=2, padding=0, dilation=1, ceil mode=False)
    (8): Conv1d(128, 256, kernel size=(5,), stride=(1,), padding=(2,))
    (9): BatchNorm1d(256, eps=1e-05, momentum=0.1, affine=True, track running stats=True)
    (10): ReLU()
    (11): AdaptiveAvgPool1d(output size=1)
  )
  (classifier): Sequential(
    (0): Linear(in features=256, out features=128, bias=True)
    (1): Dropout(p=0.5, inplace=False)
    (2): ReLU()
    (3): Linear(in features=128, out features=4, bias=True)
  )
)
```

模型训练与预测

训练参数配置:

- 生成一个ConvCodeClassifier模型
- 使用Adam优化器, 学习率设为0.001
- 使用交叉熵损失函数
- 训练轮次设为20

```
model = ConvCodeClassifier()
model.cuda() # 移至GPU
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=0.001, # 学习率
    weight_decay=1e-4 # L2正则化
)
criterion = nn.CrossEntropyLoss(label_smoothing=0.1) # 标签平滑防止过拟合
epochs = 20 # 训练轮次
```

模型训练与预测

我们应用10,000,000个数据对模型进行训练，同时用2,000,000个数据对模型进行测试，得到最终模型后，用模型预测1,000个测试信号，得到模型的预测效果。下图为训练代码（左）与测试代码（右）

```
for epoch in range(epochs):
    model.train() # 训练模式
    # 遍历训练数据批次
    for batch_idx, (inputs, labels) in enumerate(train_loader):
        inputs = inputs.cuda() # 数据移至GPU
        labels = labels.cuda()
        labels = labels.to(torch.long) # 转换为长整型
        outputs = model(inputs) # 前向传播
        loss = criterion(outputs, labels) # 计算损失
        optimizer.zero_grad() # 清空梯度
        loss.backward() # 反向传播
        optimizer.step() # 更新参数

    # 记录训练损失
    loss_value = loss.detach().cpu().item()
    epoch_losses_training.append(loss_value)
```

```
# 预测阶段
for j, (images, labels) in enumerate(test_loader1):
    images = images.cuda()
    labels = labels.cuda()
    with torch.no_grad(): # 不计算梯度
        outputs = best_model(images)
        _, preds = outputs.max(1)
    labels = labels.to(torch.long)

    # 计算测试损失
    loss_test += criterion(outputs, labels)
    total = y_prediction.size(0)

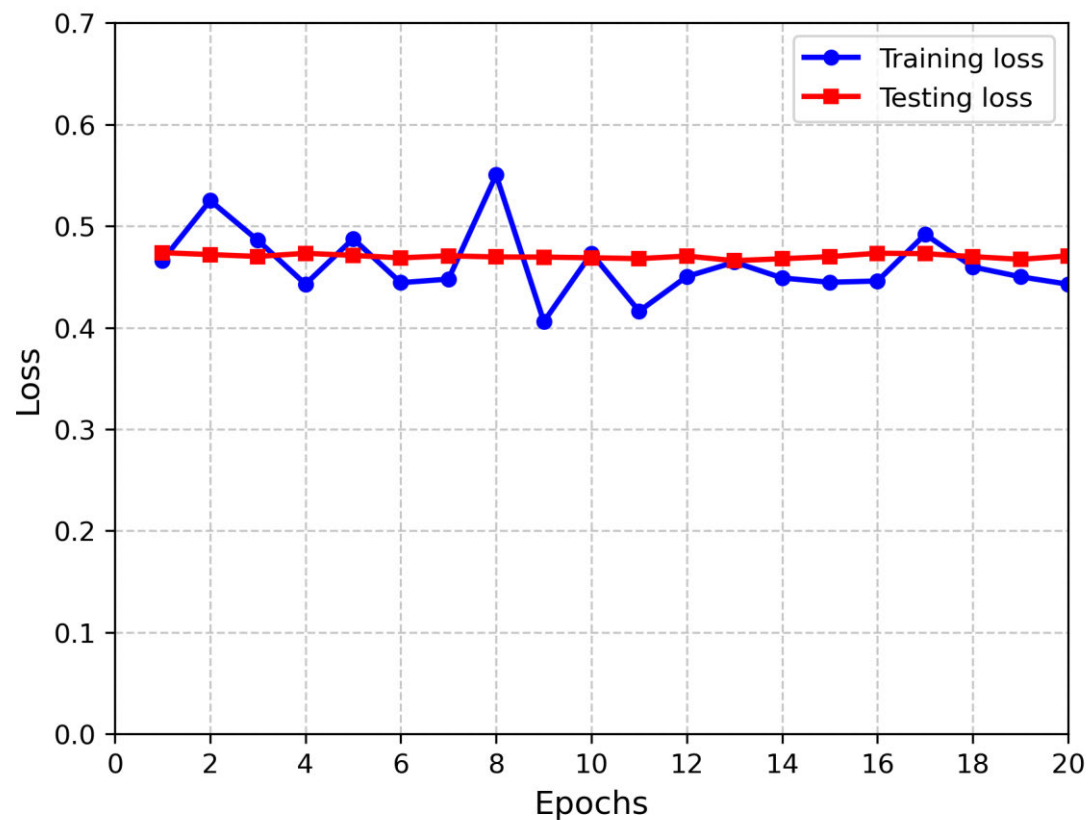
    # 计算准确率
    accuracy += (preds == labels).sum().item()

    # 保存预测结果
    if j < 1000 // 64:
        predictions[64 * j:64 * (j + 1)] = preds
    else:
        predictions[64 * j:] = preds

# 计算平均预测损失和准确率
accuracy = accuracy / len(X_prediction)
accuracies.append(accuracy)
loss_test = loss_test / (len(X_prediction)//64)
```

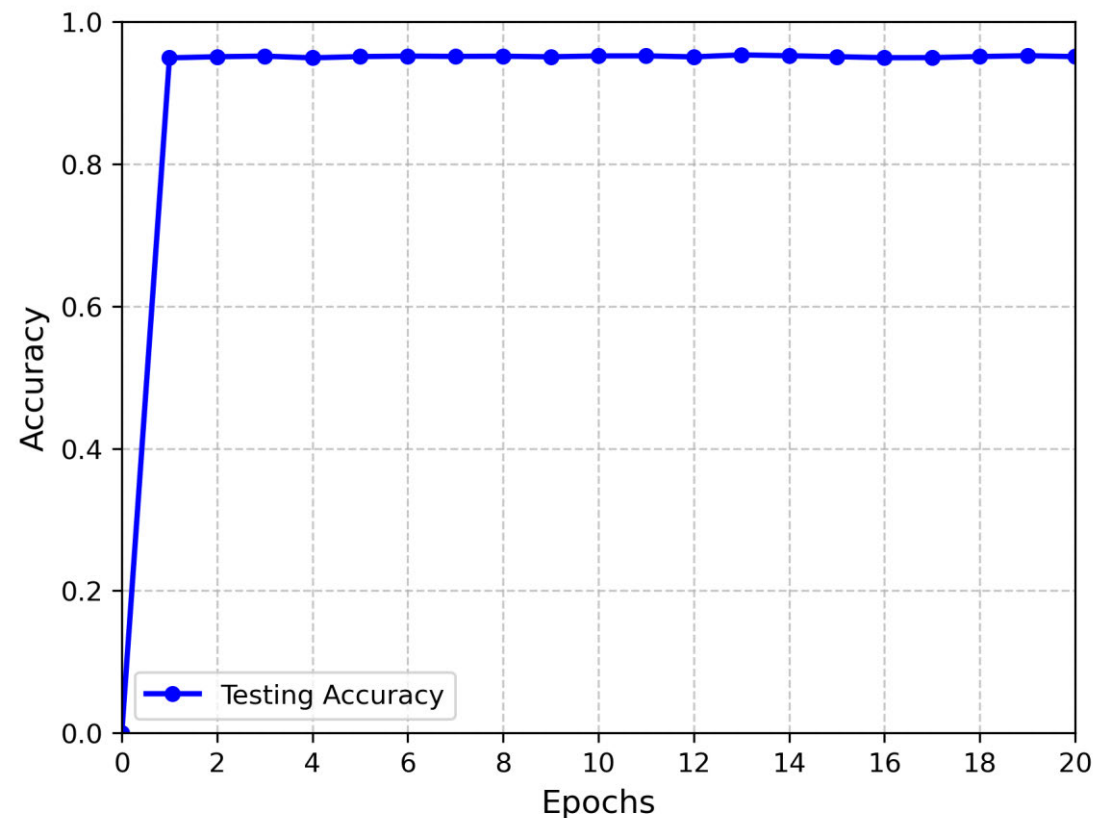

模型损失函数和正确率

Training and Testing Loss Curve



此图表明：随循环次数的增加，训练集损失函数逐渐收敛，但测试集损失函数基本保持稳定

Testing Accuracy Curve

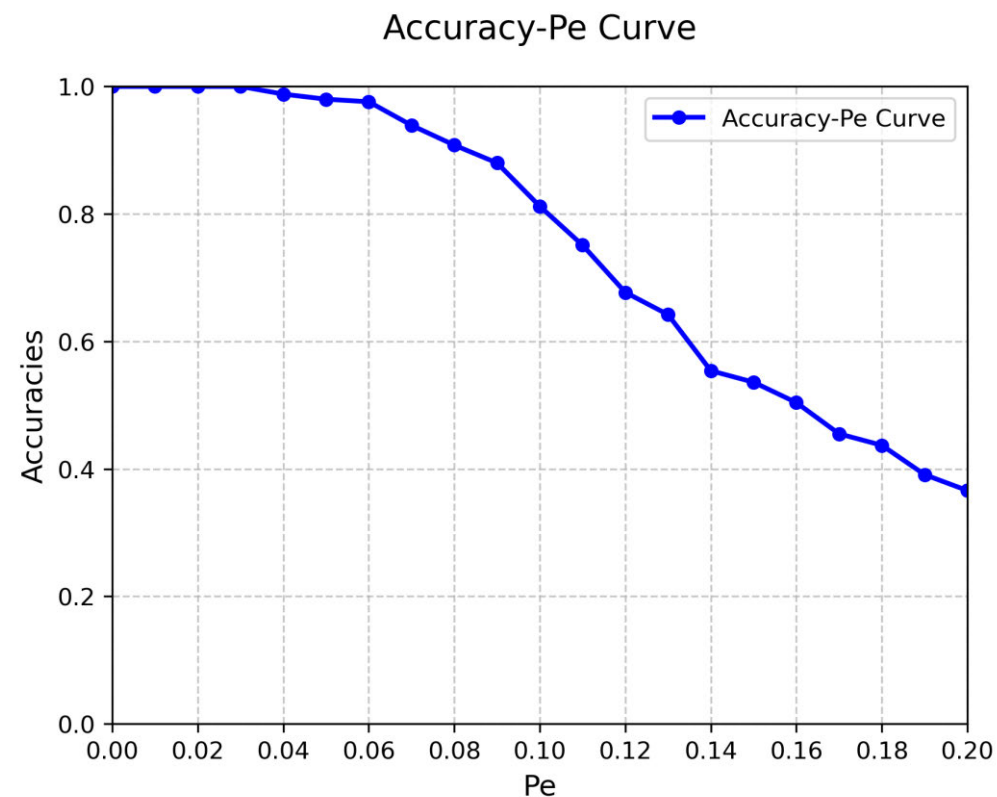


此图表明：随着训练次数增加，正确率逐渐稳定于95%~95.5%

不同误码率下，测试信号正确率

我们给出在不同误码率情况下，模型预测集正确率情况。
可以看出：

- ① 误码率为0%-3%时，测试信号的正确率为100%，可以几乎完全正确识别卷积码的生成多项式类型
- ② 误码率为4%-8%时，测试信号的正确率为90%以上，能够非常准确地识别卷积码的生成多项式类型
- ③ 误码率为9%-16%时，测试信号的正确率为50%-90%，并且随着误码率的增加快速下降，此时已逐渐超过了模型的识别能力
- ④ 当误码率继续增加时，测试信号的正确率已不足一半，模型逐渐失去预测能力



误码率	0%	1%	2%	3%	4%	5%	6%
预测集正确率	100%	100%	100%	100%	98.8%	98.0%	97.6%
误码率	7%	8%	9%	10%	11%	12%	13%
预测集正确率	93.9%	90.8%	88.0%	81.2%	75.1%	67.7%	64.2%
误码率	14%	15%	16%	17%	18%	19%	20%
预测集正确率	55.4%	53.6%	50.4%	45.5%	43.7%	39.1%	36.6%

Part 04

四、总结

神经网络模型



成功搭建一个CNN网络模型，并有效识别给定信号的卷积码生成多项式

文件输出



成功生成测试集预测结果、模型结构及模型参数量、训练时间等文件

Part 05

五、参考文献

参考文献

- [1] J. F. Sun. L. L. Zhang. M. Y. Liu. Convolutional Neural Networks for Convolutional Code Detection[J]. Journal of Communications and Information Networks, 2020, 5(2): 345-356.
- [2] Y. C. Yu. C. Y. Huang. D. J. Huang. Convolutional Neural Networks for Convolutional Code Decoding[J]. IEEE Transactions on Communications, 2022, 70(6): 1234-1245.
- [3] A. N. Savinov. Neural networks for error correction of convolutional codes[J]. Radioelectronics and Communications Systems, 2023, 66(3): 209-216.
- [4]李华, 王磊.基于卷积神经网络的中文命名实体识别研究 [J]. 中文信息学报, 2019, 33 (5): 45-52.
- [5]王敏, 刘洋.人工智能中的卷积神经网络综述 [J]. 自动化学报, 2025, 51 (5): 1001-1012.
- [6]吴晓云.算术编码算法在图像压缩中的研究[J].计算机与数字工程,2017,45(09):1863-1865.



谢谢大家

汇报小组：

时间：

2025